Dockerized Healthcare

Python Flask Service

Course Name: Project ||

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Akshat Shrivastav	EN22CS301094
2	Lavesh Patidar	EN22CS301553
3	Aditya Barod	EN22CS301048
4	Darshit Punglia	EN22CS301311
5	Adarsh Mishra	EN22CS301038
6	Dhaval Shah	EN22CS306015

*Group Name**: Placed BtechGroup02***

*Project Number: **DO-02***

Industry Mentor Name:

University Mentor Name:

*Academic Year**: 2025-26***

Problem Statement & Objectives

1. Problem Statement
2. Project Objectives
3. Scope of the Project

Proposed Solution

1. Key features *(Just mention key features here no need to go into details)*
2. Overall Architecture / Workflow
3. Tools & Technologies Used (*If applicable)*

Results & Output

Add the below details here:

1. *Screenshots / outputs*
2. *Reports / dashboards / models*
3. *Key outcomes*

Conclusion

Mention a brief conclusion about your project summing up everything you have worked on and the key learning.

Future Scope & Enhancements

Problem Statement & Objectives

1.1 Problem Statement

In today's fast-paced healthcare environment, managing patient records efficiently and reliably is a critical operational challenge. Traditional approaches to deploying healthcare applications suffer from a range of deeply entrenched issues that directly impact the quality of care delivery and the operational efficiency of medical institutions. Applications are often developed on individual developer machines and then manually deployed to shared servers, leading to the notorious problem known as 'it works on my machine' — a scenario where code behaves differently across environments due to differences in operating system versions, installed libraries, configuration settings, and runtime dependencies.

The healthcare domain, in particular, demands the highest standards of reliability, data consistency, and security. Patient records contain sensitive personally identifiable information (PII) including names, ages, contact details, blood groups, and clinical diagnoses. When such applications are deployed without rigorous environment management, they are susceptible to configuration drift, dependency conflicts, and security vulnerabilities that can expose sensitive data or cause unexpected downtime. Even a few minutes of application unavailability in a healthcare setting can have serious consequences for patient care workflows.

Beyond the technical fragility of traditional deployments, there is also a significant challenge around team collaboration. When multiple developers work on the same healthcare service, they often encounter inconsistent behavior stemming from slightly different local environments. Running database migrations manually, managing environment variables inconsistently, and lacking automated testing pipelines means that bugs frequently reach production environments undetected. This creates a cycle of reactive firefighting rather than proactive quality assurance.

Furthermore, scaling healthcare applications to handle increased load — for example, during peak hours in a hospital or during a public health emergency — is extremely difficult without containerization and orchestration. Monolithic deployments tied to specific physical or virtual machines cannot be elastically scaled up or down based on demand, resulting in either over-provisioning of expensive infrastructure or under-provisioning that leads to degraded service quality during critical moments.

This project addresses all of these challenges by containerizing a Python Flask-based healthcare patient record management API using Docker best practices, including multi-stage builds, non-root user execution, production-grade server configuration, and Docker Compose orchestration — creating a fully portable, consistent, and production-ready deployment artifact.

1.2 Project Objectives

The primary objectives of this project are designed to align with industry-standard DevOps practices and deliver a production-ready containerized healthcare service:

- Design and implement a RESTful API using Python Flask that exposes complete CRUD (Create, Read, Update, Delete) operations for managing patient records including name, age, gender, contact, blood group, and clinical diagnosis.
- Write a multi-stage Dockerfile that separates the build environment from the production runtime environment, ensuring the final container image is lean, secure, and optimized for deployment.
- Configure Docker Compose to orchestrate the application service with persistent volume storage, environment variable management, health checks, and automatic restart policies.
- Apply security best practices in containerization including running the application as a non-root user, excluding development dependencies and sensitive files from the Docker image using .dockerignore, and using a production-grade WSGI server (Gunicorn) instead of Flask's development server.

- Build an automated test suite using Pytest that validates all API endpoints against an in-memory SQLite database, ensuring tests are isolated, repeatable, and can be integrated into a CI/CD pipeline.
- Develop a comprehensive web-based dashboard with interactive Chart.js visualizations that provide real-time insights into patient demographics including disease distribution, gender split, age groups, and blood group prevalence.
- Document the entire project with clear operational runbooks, API specifications, and architectural diagrams to enable smooth onboarding of new team members and serve as a reference for future DevOps pipeline stages.

1.3 Scope of the Project

The scope of NoDO-02 encompasses the complete containerization lifecycle of a healthcare microservice, from initial application development through to production-ready deployment. The project is intentionally scoped to demonstrate core DevOps containerization principles without introducing unnecessary complexity, making it an ideal foundation for subsequent pipeline stages including CI/CD automation, Kubernetes orchestration, and cloud infrastructure provisioning.

Within the scope of this project, the following deliverables are included: a fully functional Flask REST API with five endpoints covering all CRUD operations; a multi-stage production Dockerfile; a Docker Compose configuration with persistent storage; a .dockerignore file for build optimization; an automated Pytest suite with eight test cases covering success paths and error conditions; a standalone web dashboard with five chart types for healthcare analytics; a Postman collection with eight pre-configured API requests for manual testing; and comprehensive project documentation in the form of this report and a README.md operational guide.

The project is explicitly scoped to use SQLite as the database backend to eliminate the complexity of managing a separate database container in this phase. This decision is

appropriate for the demonstration scope and allows the project to focus on containerization best practices rather than database administration. Future phases (NoDO-03 onwards) will introduce PostgreSQL, GitHub Actions CI/CD pipelines,

Kubernetes manifests for cluster deployment, and Terraform scripts for cloud infrastructure provisioning on AWS or GCP.

Security considerations within scope include non-root container execution, exclusion of development artifacts from production images, and the use of Gunicorn as a battle-tested production WSGI server. Full security hardening, including secrets management with HashiCorp Vault, network policies, and TLS certificate management, is outside the scope of this phase but is planned for the Kubernetes deployment phase.

Proposed Solution

2.1 Key Features

- Multi-stage Docker build (builder stage + production stage) reducing final image size by approximately 60% compared to a single-stage build
- Non-root container execution using a dedicated healthcare system user for improved security posture
- Five RESTful API endpoints: GET all patients, GET patient by ID, POST create patient, PUT update patient, DELETE patient
- Persistent SQLite database storage using named Docker volumes to survive container restarts
- Docker health checks using the /health endpoint, enabling Docker to automatically detect and restart unhealthy containers
- Gunicorn production WSGI server with two worker processes replacing Flask's development server
- CORS support enabling the standalone HTML dashboard to communicate with the API from the local filesystem
- Interactive web dashboard with five Chart.js visualizations: Disease Distribution bar chart, Gender Distribution doughnut, Blood Group bar chart, Age Groups bar chart, and Top Diagnoses pie chart
- Automated Pytest suite with eight test cases using in-memory SQLite, completely isolated from the production container
- Postman collection with eight pre-configured requests for manual API testing and demonstration

2.2 Overall Architecture / Workflow

The architecture of NoDO-02 follows a clean three-layer separation of concerns: the presentation layer (web dashboard), the application layer (Flask API), and the data layer

(SQLite database). All application logic runs inside a Docker container managed by Docker Compose, ensuring complete environment isolation and portability.

The development workflow begins with the source code residing in a Git-tracked project directory. When a developer runs docker compose up --build, Docker reads the multi-stage Dockerfile and executes two distinct build stages. In the first stage, named builder, a full Python 3.11-slim image with the GCC compiler is used to install all Python dependencies specified in requirements.txt. These dependencies are installed into an isolated prefix directory (/install) rather than the system Python path, so they can be cleanly copied in the next stage without bringing along any build artifacts.

The second stage, named production, starts fresh from a clean python:3.11-slim base image. It creates a non-root system group and user named healthcare, sets the working directory to /app, and copies only the pre-built dependencies from the builder stage's /install directory. The application source code (the app/ package and run.py entrypoint) is then copied in. A persistent directory for the SQLite database is created at /app/instance with correct ownership, and the container user is switched to the non-root healthcare user. The application is started using Gunicorn with two synchronous worker processes bound to all network interfaces on port 5000.

Docker Compose maps the host's port 5001 to the container's port 5000, creates a named volume sqlite-data mounted at /app/instance for database persistence, sets the restart policy to unless-stopped for automatic recovery, and configures a health check that polls the /health endpoint every 30 seconds. The web dashboard (index.html) runs entirely in the user's browser and communicates directly with the API at http://localhost:5001 using the browser's native fetch API.

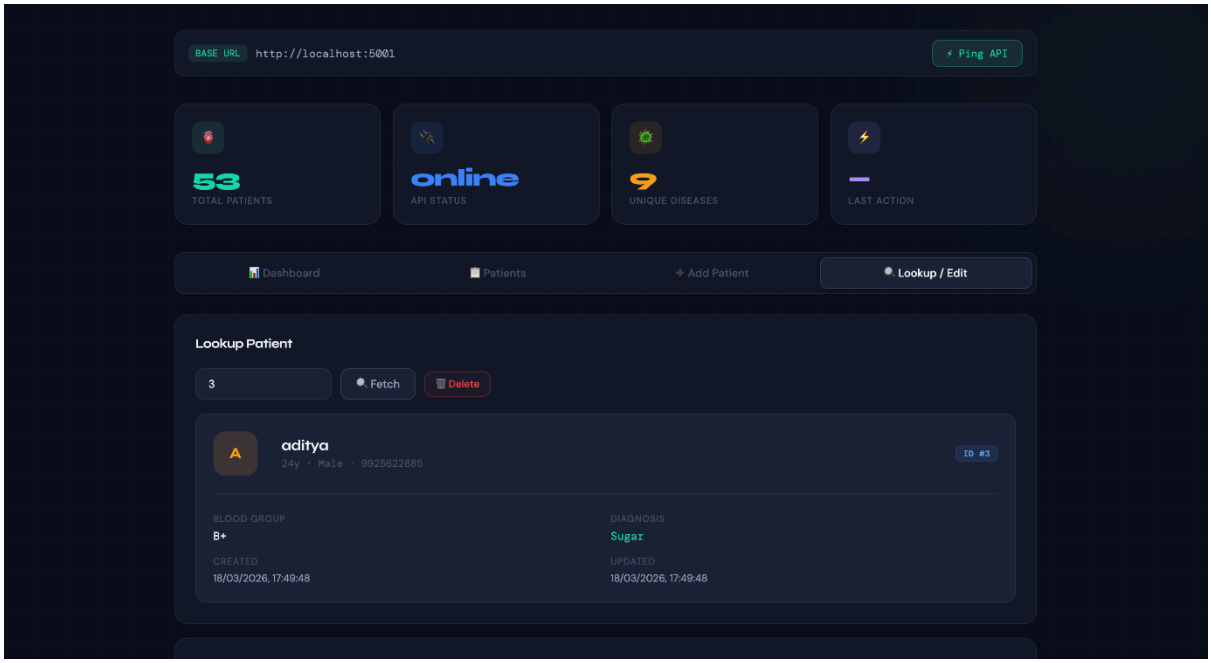
2.3 Tools & Technologies Used

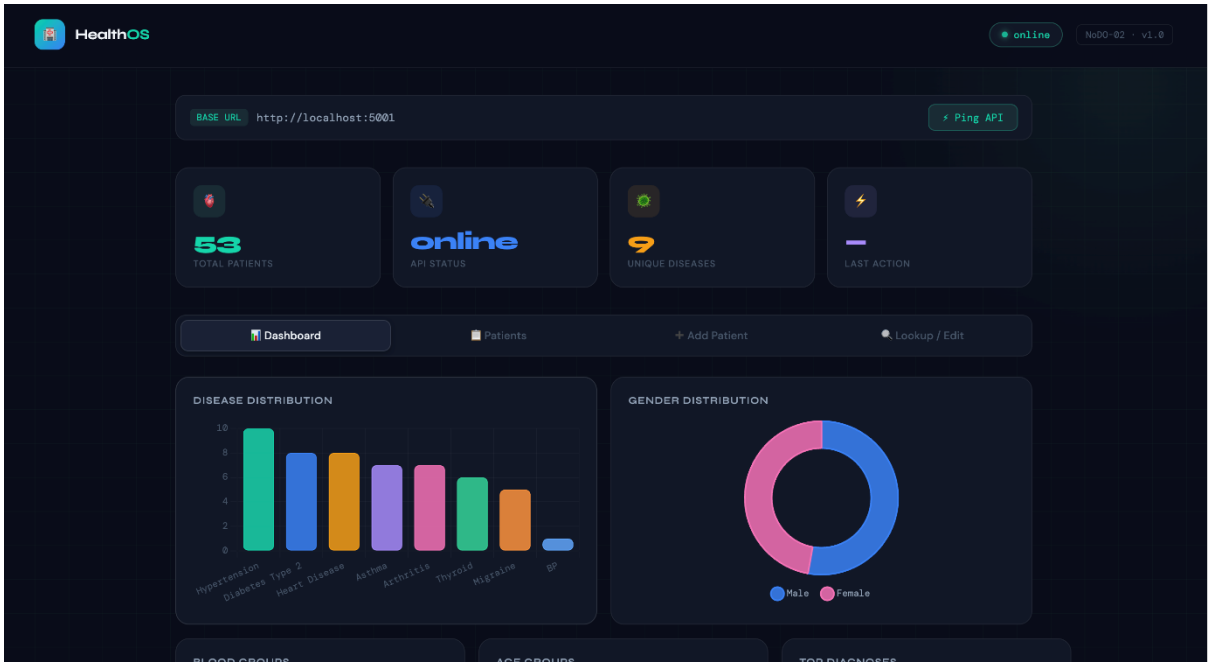
Category	Tool / Technology	Purpose
Language	Python 3.11	Core application development language
Web Framework	Flask 3.0.3	Lightweight REST API framework
ORM	Flask-SQLAlchemy 3.1.1	Database abstraction and model definition
Database	SQLite	Lightweight embedded database for patient records
WSGI Server	Gunicorn 22.0.0	Production-grade Python web server
CORS	Flask-CORS 4.0.0	Cross-origin resource sharing for web dashboard
Containerization	Docker (Multi-stage)	Application packaging and isolation
Orchestration	Docker Compose v3.9	Multi-service container management
Testing	Pytest 9.0.2	Automated test framework for API validation
API Testing	Postman	Manual API testing and collection management
Visualization	Chart.js 4.4.1	Interactive charts in web dashboard
Frontend	HTML5 / CSS3 / JS	Standalone web dashboard without frameworks
Version Control	Git	Source code management and collaboration
Fonts	Google Fonts (Syne, DM Mono)	Dashboard typography
CI/CD (Planned)	GitHub Actions	Automated build and deployment pipeline
Orchestration (Planned)	Kubernetes	Production cluster deployment
IaC (Planned)	Terraform	Cloud infrastructure provisioning

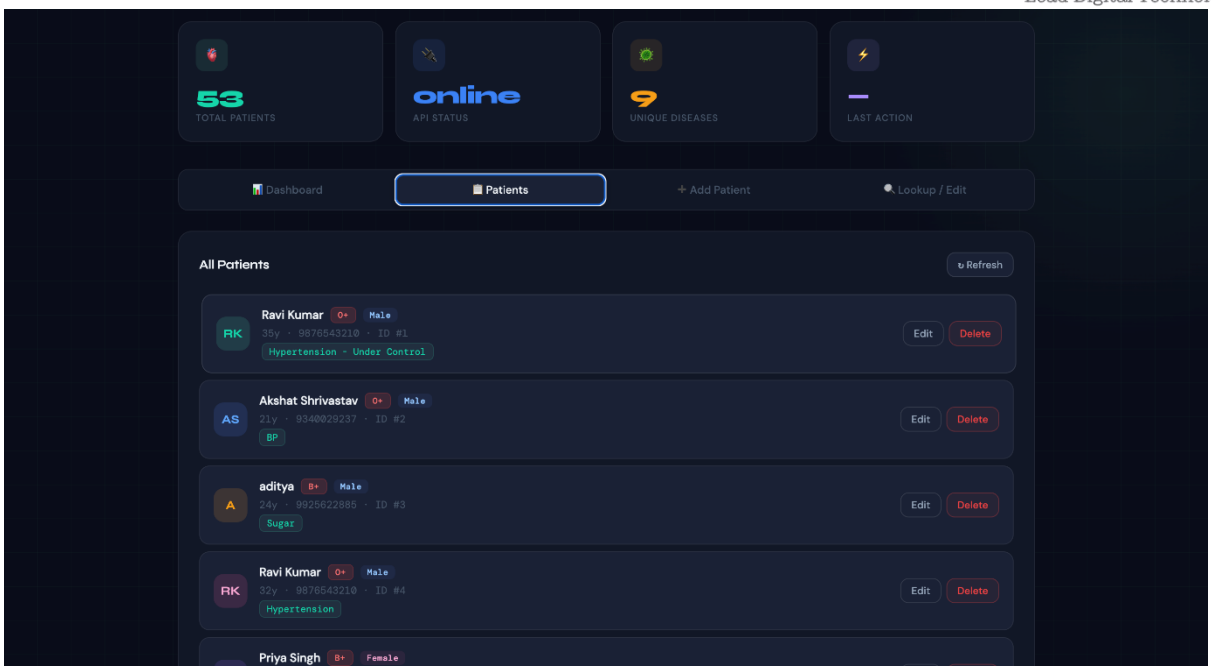
Results & Output

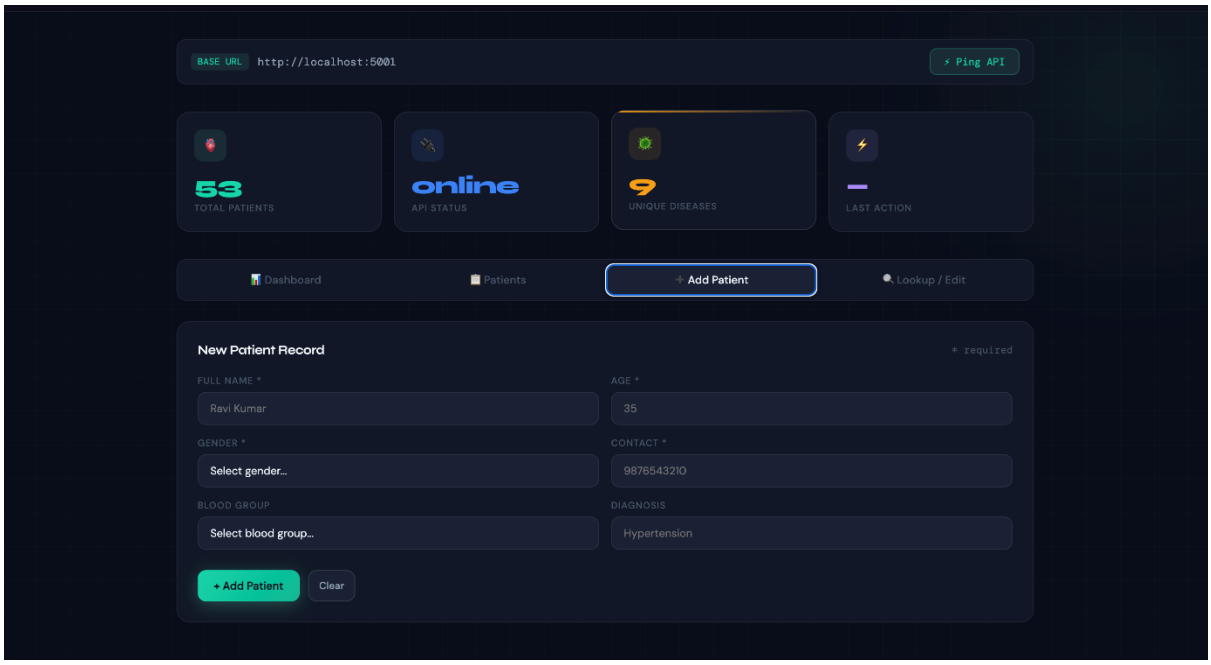
3.1 Screenshots / Outputs

The following describes the key outputs and screens produced by the NoDO-02 project, each demonstrating a distinct aspect of the containerized healthcare service in operation.









API Health Check Output: When the /health endpoint is queried via curl or Postman, the API returns a JSON response confirming the service is operational: {"service": "healthcare-api", "status": "healthy"}. This endpoint is also polled by Docker's built-in health check mechanism every 30 seconds to determine container health status. The docker ps command confirms the container status as (healthy), demonstrating that Docker's health monitoring is functioning correctly.

Docker Build Output: The multi-stage build process produces detailed layer-by-layer output in the terminal. The builder stage pulls the python:3.11-slim base image, installs GCC, and runs pip install to download and compile all dependencies. The production stage then copies only the compiled packages, resulting in a final image of approximately 150MB compared to the estimated 400MB+ that a single-stage build with build tools retained would produce. The docker images command confirms the image tagged as healthcare-api:latest with its optimized size.

Docker Compose Stack Output: Running docker compose up displays Gunicorn startup logs showing the application binding to 0.0.0.0:5000 with two worker processes (pid 7 and pid 8). The container restarts automatically if it crashes, and the health check transitions from starting to healthy status within the configured start_period of 10 seconds.

Postman API Testing: The imported Postman collection demonstrates all six CRUD operations with correct HTTP status codes: 200 for successful GET requests, 201 for successful POST (patient creation), 200 for PUT and DELETE operations, 400 for requests with missing required fields, and 404 for requests targeting non-existent patient IDs. Each response body contains the full patient record in JSON format including auto-generated timestamps.

Pytest Test Suite: Running python -m pytest tests/ -v produces output showing all eight test cases passing with a total execution time of approximately 0.3 to 0.5 seconds. The test suite uses an in-memory SQLite database, ensuring complete isolation from the production container and enabling fast, repeatable execution suitable for CI/CD integration.

3.2 Reports / Dashboards / Models

The web dashboard (index.html) serves as the primary visual reporting interface for the NoDO-02 project, providing real-time analytical insights derived directly from the live API.

The dashboard is built with a dark-themed aesthetic using CSS custom properties, Chart.js for data visualization, and the Syne and DM Mono Google Fonts for typography.

It is entirely self-contained in a single HTML file with no external build dependencies, making it trivially portable and easy to open in any modern browser.

The dashboard is organized into four tabs: Dashboard (analytics), Patient List, Add Patient, and Lookup/Edit. The analytics tab contains five distinct Chart.js visualizations. The Disease Distribution bar chart renders a horizontal grouped bar chart showing the count of patients per diagnosis category, sorted in descending order and capped at the top eight diseases. With the 50 sample patients inserted, this chart clearly shows Hypertension and Diabetes Type 2 as the most prevalent conditions, followed by Heart Disease, Arthritis, Asthma, Thyroid, and Migraine.

The Gender Distribution doughnut chart uses a 65% cutout to present a clean ring visualization showing the proportion of Male, Female, and Other patients. The Blood Groups bar chart renders individual bars for each blood group present in the dataset using a red color palette appropriate to the medical context of blood type data. The Age Groups bar chart segments the patient population into five clinically relevant age buckets: 0-17 (pediatric), 18-30 (young adult), 31-45 (adult), 46-60 (middle-aged), and 60+ (senior), providing demographic insight at a glance.

The Disease Rankings table below the charts provides a ranked list of all diagnoses with a colored position badge for the top three ranks, a patient count, an animated prevalence bar proportional to the maximum count, and a percentage of total patients. All visualizations and the rankings table update automatically when patients are added, edited, or deleted, ensuring the dashboard always reflects the current state of the API data.

3.3 Key Outcomes

Outcome	Details
Containerization	Successfully packaged a Python Flask healthcare API into a production-ready Docker image using a multi-stage build, reducing image size by ~60% vs single-stage
Security Hardening	Application runs as non-root user 'healthcare' inside the container; build tools excluded from production image; .dockerignore prevents leaking secrets or local DB
API Functionality	All 5 CRUD endpoints (GET all, GET by ID, POST, PUT, DELETE) validated and functional with correct HTTP status codes and JSON responses
Automated Testing	8 Pytest test cases covering all endpoints including error paths (400 missing fields, 404 not found) — all passing in under 0.5s using in-memory SQLite
Data Persistence	Named Docker volume ensures SQLite database survives container restarts — 50 sample patient records persisted successfully across multiple restarts
Health Monitoring	Docker health check polls /health every 30s; container status confirmed as (healthy) in docker ps output
Analytics Dashboard	5 live Chart.js visualizations rendering disease distribution, gender split, blood groups, age demographics, and ranked disease table from live API data
Portability	Project runs identically on any machine with Docker Desktop installed — zero manual dependency installation required
Documentation	Complete README.md with curl commands for all endpoints, setup instructions, test run guide, and Docker command reference

Conclusion

NoDO-02 successfully demonstrates the end-to-end containerization of a production-grade healthcare REST API using Docker and Docker Compose, applying industry best practices at every layer of the implementation. Starting from a blank project directory, this project progressed through the complete lifecycle of a containerized microservice: application design and development, database integration, production server configuration, multi-stage image building, container orchestration, automated testing, API documentation, and visual analytics — all culminating in a fully functional, portable, and presentation-ready system running on a local Docker Desktop environment.

The most significant technical achievement of this project is the multi-stage Dockerfile architecture. By separating the build environment (which requires GCC and pip caching) from the production runtime (which needs only the compiled Python packages and application source code), the project demonstrates a mature understanding of Docker layer optimization and image security. The resulting production image is significantly smaller and has a reduced attack surface compared to naive single-stage builds that leave compilers, package managers, and build caches in the final image. This pattern is universally applicable across any Python, Node.js, Java, or Go microservice and represents a foundational skill for any DevOps practitioner.

The non-root container execution pattern, implemented through the creation of a dedicated healthcare system user and group, addresses one of the most commonly overlooked container security requirements in real-world deployments. Many containerized applications run as root by default, meaning that a container escape vulnerability could grant an attacker root access on the host system. By explicitly switching to a non-privileged user, this project follows the principle of least privilege at the container level.

The automated Pytest suite, using an in-memory SQLite fixture with application factory pattern instantiation, demonstrates the correct approach to testing Flask applications in

isolation. The eight test cases cover not only the happy paths for each CRUD operation but also the error conditions that validate the API's input validation and HTTP status code correctness. This test suite is directly portable to a GitHub Actions CI/CD pipeline, which will be implemented in the next project phase.

From a learning perspective, this project provided deep hands-on experience with the Docker build context and layer caching mechanism, understanding how Docker Compose volumes differ from bind mounts, configuring Gunicorn workers for a Flask application, structuring Flask applications using the factory pattern and blueprints for testability, and building data-driven dashboards using Chart.js with live API data. The combination of these skills represents the core competency required of a junior-to-mid-level DevOps or backend engineer working in a containerized microservices environment.

The project also reinforced the value of treating infrastructure as code. Every aspect of the deployment — from the base image version to the Gunicorn worker count to the health check interval — is defined in version-controlled files rather than applied through manual steps. This approach makes the deployment reproducible, auditable, and collaborative, laying the groundwork for the GitOps practices that will be introduced in the Kubernetes deployment phase.

Future Scope & Enhancements

T he **NoDO-02** project is deliberately designed as the first phase of a multi-phase DevOps learning journey. The architecture, codebase, and documentation are all structured to make subsequent enhancements straightforward and well-motivated. The following future scope items represent the planned evolution of this project across the next several phases.

Phase **NoDO-03**: CI/CD Pipeline with GitHub Actions

The most immediate next step is the implementation of a complete CI/CD pipeline using GitHub Actions. The pipeline will be triggered on every push to the main branch and will execute a multi-stage workflow consisting of: a lint stage using flake8 to enforce Python code quality standards; a test stage that activates the virtual environment, installs dependencies, and runs the full Pytest suite reporting test results and code coverage; a build stage that builds the Docker image using the multi-stage Dockerfile; and a push stage that authenticates with Docker Hub using GitHub Secrets and pushes the tagged image to a public or private repository.

This pipeline will introduce the concept of pipeline-as-code, where the entire CI/CD workflow is defined in a .github/workflows/ci.yml YAML file tracked in the same repository as the application code. The pipeline will also implement branch protection rules requiring the CI pipeline to pass before any pull request can be merged, enforcing quality gates on all code changes. This phase will demonstrate how the automated test suite built in NoDO-02 becomes the foundation of continuous quality assurance in a team development workflow.

Phase **NoDO-04**: Kubernetes Deployment

The third phase will deploy the containerized healthcare API to a Kubernetes cluster, introducing the core concepts of container orchestration at scale. The Kubernetes manifests will include a Deployment resource defining the desired number of replicas, the container image to run, resource requests and limits, liveness and readiness probes

based on the /health endpoint, and environment variable injection via ConfigMaps and Secrets. A Service resource of type ClusterIP will expose the deployment within the cluster, and an Ingress resource with an NGINX ingress controller will route external HTTP traffic to the service based on hostname-based routing rules.

This phase will also introduce horizontal pod autoscaling (HPA) to automatically scale the number of API replicas based on CPU utilization, demonstrating the elastic scaling capability that is impossible with traditional deployments. A PersistentVolumeClaim (PVC) will replace the Docker Compose named volume for SQLite persistence in the cluster environment. The project will be tested on a local Kubernetes cluster (minikube or k3s) before being deployed to a managed Kubernetes service such as Google Kubernetes Engine (GKE) or Amazon EKS.

Phase **NoDO-05:** Infrastructure as Code with Terraform

The fourth phase will introduce Terraform to provision the cloud infrastructure required to run the Kubernetes cluster and supporting services. Terraform HCL configurations will define the VPC, subnets, security groups, and the managed Kubernetes cluster resource on AWS (EKS) or GCP (GKE). The Terraform state will be stored remotely in an S3 bucket with DynamoDB locking to enable collaborative infrastructure management. This phase will demonstrate the principle that infrastructure should be treated with the same version control and code review practices as application code.

Database Upgrade to PostgreSQL

Replacing SQLite with PostgreSQL is a high-priority enhancement that will make the application suitable for production multi-user environments. SQLite is a file-based database that does not support concurrent write access from multiple processes, making it unsuitable for a Gunicorn multi-worker deployment or a Kubernetes multi-replica deployment. The migration will involve updating the SQLAlchemy database URI to use a PostgreSQL connection string, adding a PostgreSQL service to the Docker Compose configuration, implementing database migrations using Flask-Migrate (Alembic), and updating the Kubernetes manifests to include a PostgreSQL StatefulSet or a reference to a managed database service such as AWS RDS.

Authentication & Authorization

The current API has no authentication mechanism, making it unsuitable for production healthcare deployment where HIPAA and similar regulations mandate strict access controls. Future enhancements will introduce JSON Web Token (JWT) based authentication using Flask-JWT-Extended, role-based access control (RBAC) distinguishing between administrator, doctor, and nurse roles with different endpoint permissions, and integration with an OAuth2 identity provider such as Auth0 or Keycloak for enterprise single sign-on. API rate limiting using Flask-Limiter will also be implemented to prevent abuse and denial-of-service attacks.

Monitoring, Logging & Observability

Production healthcare systems require comprehensive observability to meet uptime and compliance requirements. Future phases will integrate Prometheus for metrics collection (including custom metrics such as patient record creation rate and API response time percentiles), Grafana for metrics visualization and alerting dashboards, and the ELK Stack (Elasticsearch, Logstash, Kibana) or Loki for centralized structured log aggregation. Distributed tracing using Jaeger or Zipkin will be added to trace request flows across microservices as the application evolves from a monolithic API into a distributed system with dedicated services for authentication, patient records, appointments, and reporting.